

DSC 540 - Data Preparation

Weeks 5 & 6 - Term Project: Milestone 2

Manoj K Kola

Cleaning/Formatting Flat File Source

Python Libraries for Data Analysis

This code snippet demonstrates the import of two fundamental Python libraries widely used in data analysis and numerical computing:

1. **pandas:** Imported as `pd`, pandas is a powerful library for working with structured data, such as spreadsheets or CSV files. It provides the `DataFrame` and `Series` data structures, which allow for easy manipulation, analysis, and cleaning of tabular data.
2. **numpy:** Imported as `np`, NumPy is a library for numerical operations in Python. It provides support for multi-dimensional arrays, mathematical functions, and efficient computation, which are often essential for data analysis and scientific computing.

```
In [55]: # Import pandas for working with data tables (DataFrames)
import pandas as pd

# Import numpy for numerical operations and calculations
import numpy as np
```

Load and Preview the Dataset

1. **Specify File Path:** The path to the CSV file containing city population data is stored in the variable `file_path`.
2. **Read CSV File:** The dataset is read into a pandas DataFrame named `df` using `pd.read_csv()`. This allows us to work with the data in a structured table format.
3. **Preview Data:** The `df.head()` function displays the first few rows of the DataFrame, giving a quick look at the data columns and sample values.

```
In [56]: # Specify the file path for the CSV file
file_path = "Total_Population_By_City_Acs_2015_5_E_AgeSex.csv"

# Read the CSV file into a pandas DataFrame
df = pd.read_csv(file_path)
```

```
# Display the first few rows of the dataset
df.head()
```

Out[56]:

	index	Table_Id	City	Summary_Level	Place_Fips	Geo_Id	State	State_Fips
0	0	100001	Coffman Cove	160	16360	16000US0216360	AK	2
1	1	100002	Cohoe	160	16420	16000US0216420	AK	2
2	2	100003	Cold Bay	160	16530	16000US0216530	AK	2
3	3	100004	Coldfoot	160	16630	16000US0216630	AK	2
4	4	100005	College	160	16750	16000US0216750	AK	2

Step #1: Replace / Clean Column Headers

Description: Column names are cleaned by converting them to uppercase and replacing spaces with underscores for consistency and easier access.

Cleaning and Previewing DataFrame Columns

The following code snippet ensures that the DataFrame's column names are consistent and easy to work with. This is a common preprocessing step before performing any analysis.

1. Clean column names:

- `str.strip()` removes any leading or trailing spaces.
- `str.upper()` converts all column names to uppercase for uniformity.
- `str.replace(" ", "_")` replaces spaces with underscores, making column names easier to reference in code.

2. Display updated column names: Verify that the changes have been applied correctly.

3. Preview the DataFrame: Display the first few rows to ensure the data looks correct and ready for analysis.

```
In [69]: # Clean the column names for consistency
df.columns = (
    df.columns
    .str.strip()           # Remove leading and trailing spaces
    .str.upper()           # Convert column names to uppercase
    .str.replace(" ", "_")# Replace spaces with underscores
)

# Display the updated column names
df.columns
```

```
Out[69]: Index(['INDEX', 'TABLE_ID', 'CITY', 'SUMMARY_LEVEL', 'PLACE_FIPS', 'GEO_ID',
               'STATE', 'STATE_FIPS', 'TOTAL_POPULATION'],
              dtype='object')
```

```
In [70]: # Preview the first few rows of the dataframe
df.head()
```

	INDEX	TABLE_ID	CITY	SUMMARY_LEVEL	PLACE_FIPS	GEO_ID	STATE	STA
0	0	100001	Coffman Cove	160	16360	16000US0216360	AK	
1	1	100002	Cohoe	160	16420	16000US0216420	AK	
2	2	100003	Cold Bay	160	16530	16000US0216530	AK	
3	3	100004	Coldfoot	160	16630	16000US0216630	AK	
4	4	100005	College	160	16750	16000US0216750	AK	

Step #2: Handle Missing Values

Description: Missing numeric values are replaced with 0 to prevent errors during analysis.

Handling Missing Values and Previewing the Data

In this step, we address missing data in our dataset by replacing all `NaN` values with `0`. This ensures that subsequent analysis or calculations are not affected by empty values.

After filling the missing values, we use `df.head()` to display the first few rows of the dataframe. This allows us to quickly verify that the missing values have been handled correctly and to preview the dataset structure.

```
In [58]: # Replace all missing (NaN) values in the dataframe with 0
df = df.fillna(0)

# Display the first few rows of the dataframe to preview the data
df.head()
```

	INDEX	TABLE_ID	CITY	SUMMARY_LEVEL	PLACE_FIPS	GEO_ID	STATE	STA
0	0	100001	Coffman Cove	160	16360	16000US0216360	AK	
1	1	100002	Cohoe	160	16420	16000US0216420	AK	
2	2	100003	Cold Bay	160	16530	16000US0216530	AK	
3	3	100004	Coldfoot	160	16630	16000US0216630	AK	
4	4	100005	College	160	16750	16000US0216750	AK	

Step #3: Fix Casing and Inconsistent Text Values

Description: City and state names are standardized using title case to ensure consistency.

Data Cleaning: Standardizing City and State Columns

This section of the code checks if the dataframe contains the `CITY` and `STATE` columns and standardizes their formatting:

- **CITY Column:** Converts all city names to title case (e.g., "new york" → "New York") to ensure consistent capitalization.
- **STATE Column:** Converts all state abbreviations to uppercase (e.g., "oh" → "OH") for uniform representation.

Finally, the first few rows of the dataframe are displayed to verify that the changes have been applied correctly.

```
In [62]: # Check if the 'CITY' column exists in the dataframe
if 'CITY' in df.columns:
    # Convert city names to title case (e.g., new york -> New York)
    df['CITY'] = df['CITY'].str.title()

# Check if the 'STATE' column exists in the dataframe
if 'STATE' in df.columns:
    # Convert state values to uppercase (e.g., oh -> OH)
    df['STATE'] = df['STATE'].str.upper()

# Display the first few rows of the dataframe
df.head()
```

Out[62]:	INDEX	TABLE_ID	CITY	SUMMARY_LEVEL	PLACE_FIPS	GEO_ID	STATE	STA
0	0	100001	Coffman Cove	160	16360	16000US0216360	AK	
1	1	100002	Cohoe	160	16420	16000US0216420	AK	
2	2	100003	Cold Bay	160	16530	16000US0216530	AK	
3	3	100004	Coldfoot	160	16630	16000US0216630	AK	
4	4	100005	College	160	16750	16000US0216750	AK	

Step #4: Identify and Remove Duplicate Records

Description: Duplicate rows are identified and removed to avoid double-counting.

Data Cleaning: Removing Duplicate Rows

This section of the code removes any duplicate rows from the dataframe to ensure that each record is unique.

- **Remove Duplicates:** The `drop_duplicates()` function eliminates all duplicate rows based on all columns.

Finally, the first few rows of the dataframe are displayed to preview the cleaned data and verify that duplicates have been removed.

```
In [60]: # Remove duplicate rows from the dataframe
df = df.drop_duplicates()

# Display the first few rows to preview the cleaned data
df.head()
```

```
Out[60]:
```

	INDEX	TABLE_ID	CITY	SUMMARY_LEVEL	PLACE_FIPS	GEO_ID	STATE	ST/
0	0	100001	Coffman Cove	160	16360	16000US0216360	AK	
1	1	100002	Cohoe	160	16420	16000US0216420	AK	
2	2	100003	Cold Bay	160	16530	16000US0216530	AK	
3	3	100004	Coldfoot	160	16630	16000US0216630	AK	
4	4	100005	College	160	16750	16000US0216750	AK	

Step #5: Identify and Remove Outliers

Description: Population values that are negative or unrealistically high are removed to improve data quality.

Data Cleaning: Filtering TOTAL_POPULATION Column

This section of the code checks if the dataframe contains the `TOTAL_POPULATION` column and filters the data based on population size:

- **TOTAL_POPULATION Column:** Keeps only the rows where the population is between 50 and 1,000,000 to remove outliers and ensure meaningful data.

Finally, the first few rows of the cleaned dataframe are displayed to verify that the filtering has been applied correctly.

```
In [72]: # Check if the TOTAL_POPULATION column exists in the dataframe
if 'TOTAL_POPULATION' in df.columns:

    # Keep only rows where population is between 50 and 1,000,000
    df = df[
        (df['TOTAL_POPULATION'] >= 50) &
        (df['TOTAL_POPULATION'] <= 1_000_000)
    ]
```

```
# Display the first few rows of the cleaned dataframe
df.head()
```

Out[72]:

	INDEX	TABLE_ID	CITY	SUMMARY_LEVEL	PLACE_FIPS	GEO_ID	STATE	STA
0	0	100001	Coffman Cove	160	16360	16000US0216360	AK	
1	1	100002	Cohoe	160	16420	16000US0216420	AK	
4	4	100005	College	160	16750	16000US0216750	AK	
5	5	100006	Cooper Landing	160	17190	16000US0217190	AK	
6	6	100007	Copper Center	160	17300	16000US0217300	AK	

Step #6: Format Numeric Columns

Description: Ensures population-related columns are stored as integers for readability.

Data Cleaning: Converting Numeric Columns to Integer

This section of the code identifies all numeric columns in the dataframe and standardizes their data type:

- **Numeric Columns:** Selects all columns containing numeric values, including both floats and integers.
- **Type Conversion:** Converts all selected numeric columns to integer type to ensure consistency and remove any decimal values.

These steps help maintain uniform numeric formats across the dataframe for easier analysis.

```
In [65]: # Select all columns that contain numeric values (floats or integers)
numeric_cols = df.select_dtypes(include=['float64', 'int64']).columns

# Convert all selected numeric columns to integer type
df[numeric_cols] = df[numeric_cols].astype(int)
```

Final Cleaned Dataset (Human-Readable Output), Dataset Overview: Shape and Sample Data

This section provides a quick overview of the cleaned dataset:

- **Dataset Shape:** Prints the number of rows and columns in the final cleaned dataframe to give an idea of its size.
- **Sample Data:** Displays the first 10 rows of the cleaned dataset to visually inspect the data and verify the cleaning steps.

```
In [66]: # Print the number of rows and columns in the final cleaned dataset
print("Final cleaned dataset shape:", df.shape)

# Display the first 10 rows of the cleaned dataset
df.head(10)
```

Final cleaned dataset shape: (29564, 9)

Out[66]:	INDEX	TABLE_ID	CITY	SUMMARY_LEVEL	PLACE_FIPS	GEO_ID	STATE	ST
0	0	100001	Coffman Cove	160	16360	16000US0216360	AK	
1	1	100002	Cohoe	160	16420	16000US0216420	AK	
2	2	100003	Cold Bay	160	16530	16000US0216530	AK	
3	3	100004	Coldfoot	160	16630	16000US0216630	AK	
4	4	100005	College	160	16750	16000US0216750	AK	
5	5	100006	Cooper Landing	160	17190	16000US0217190	AK	
6	6	100007	Copper Center	160	17300	16000US0217300	AK	
7	7	100008	Cordova	160	17410	16000US0217410	AK	
8	8	100009	Covenant Life	160	17670	16000US0217670	AK	
9	9	100010	Craig	160	17740	16000US0217740	AK	

Ethical Implications of Data Wrangling

The data wrangling process involved several transformations, including column renaming, handling missing values, standardizing text case, removing duplicates, and filtering outliers. These changes were made to improve data quality, readability, and analytical usability. However, such transformations can introduce ethical risks if not handled carefully. For example, removing outliers or duplicates may unintentionally eliminate valid but rare data points, potentially leading to biased results or underrepresentation of smaller populations. Similarly, replacing missing values with defaults (such as zeros) makes assumptions that could distort interpretation if the missingness is meaningful.

This dataset is derived from U.S. Census ACS population data, which is publicly available and does not include personally identifiable information, minimizing legal and privacy concerns. As a government-provided dataset, it adheres to public data usage standards and is considered credible. However, regulatory considerations still apply to accurately represent demographic information and avoid misleading conclusions.

Assumptions were made during cleaning, such as defining acceptable population ranges. To mitigate ethical risks, conservative thresholds were used, all transformations were documented, and no irreversible changes were made without validation. Transparency in documenting each step ensures reproducibility and accountability. Overall, the data was ethically sourced, responsibly transformed, and handled in a way that minimizes harm while improving analytical reliability.

In []: